

Символни низове. Форматиране на вход/изход.

Тип string. Дефиниция и синтаксис.

В Python, както и в много други програмни езици символният низ (стринг) представлява крайна поредица от символи. За разлика от други езици в Python стринговете могат да бъдат оградени с единични или с двойни кавички.

```
In [1]: greeting1 = 'Hello, World!'
greeting2 = "Hello, World!"

print(greeting1)
print(greeting2)
```

```
Hello, World!
Hello, World!
```

Също така може да дефинираме многоредови стрингове.

```
In [2]: multiline_string1 = '''This is the first line of the string...
This is the second line of the string...
This is the third line of the string...'''

print(multiline_string1)
```

```
This is the first line of the string...
This is the second line of the string...
This is the third line of the string...
```

```
In [3]: multiline_string2 = """This is the first line of the string...
This is the second line of the string...
This is the third line of the string..."""

print(multiline_string2)
```

```
This is the first line of the string...
This is the second line of the string...
This is the third line of the string...
```

Както и в други езици, Python стринговете са масиви от символи, т.е. можем да ги обхождаме като такива.

```
In [4]: some_string = 'Hello! My name is John Doe!'

for letter in some_string:
    print(letter.capitalize(), end='')
```

```
HELLO! MY NAME IS JOHN DOE!
```

Намиране на дължина на стринг и character escaping. В Python намирането на дължината на стринг става с функцията `len()`. Когато искаме стрингът ни да съдържа специален символ (кавичка, нов ред, табулация и т.н.) използваме символът `"\"`.

```
In [5]: some_other_string = 'Hello, John! I\'m Jane'

print(some_other_string)
print(len(some_other_string))
```

```
Hello, John! I'm Jane
21
```

String slicing.

Слайсването в Python има следния вид:

```
some_sequence[\\:\\]
```

Като всеки от "параметрите" на индексацията има стойност по подразбиране - за началния индекс - 0, за крайния индекс - последния индекс на редицата, а за стъпката - 1.

```
In [6]: a = "Hello there!"

print(a[4])
print(a[4:])
print(a[4:9])
print(a[::2])
```

```
o
o there!
o the
Hlotee
```

Всеки един от тези индекси може да бъде и отрицателен.

```
In [7]: a = "General Kenobi..."

print(a[-4])

print(a[-4:-10])

print(a[-4:-10:-1])

print(a[::-2])
```

```
i
```

Обработка на стрингове.

В Python стринговете са непроменяеми (immutable). По тази причина всички функции (или по-точно методи) връщат нов стринг, вместо да модифицират оригиналния, т.е. трябва да присвоим резултата на някаква променлива.

Ще разгледаме функции (или по-точно методи) за:

- Капитализация - `.upper()/lower()`
- Махане на празни места в начало/край на стринга - `.strip()`
- Разделяне на стринга по някакъв разделител - `.split()` връща масив от стрингове
- Смяна на даден стринг / подстринг с друг - `.replace()`

```
In [8]: str1 = 'Hello World!'

# Не модифицираме оригиналния стринг, а връщаме нов такъв
str1.upper()
print(str1)

str1 = str1.upper()
print(str1)
```

```
Hello World!
HELLO WORLD!
```

```
In [9]: str1 = '  Hello, World! My name is John! '
```

```
print('str1.upper() -> ', str1.upper())

print('str1.lower() -> ', str1.lower())

print('str1.strip() -> ', str1.strip())

# По подразбиране разделителят е "\s" или space.

print('str1.split() -> ', str1.split())

print('str1.split(\'\', \') -> ', str1.split(',', ''))

print('str1.replace(\'John\', \'Jeff\') -> ', str1.replace('John', 'Jeff'))
```

```
str1.upper() ->    HELLO, WORLD! MY NAME IS JOHN!
str1.lower() ->    hello, world! my name is john!
str1.strip() ->    Hello, World! My name is John!
str1.split() ->    ['Hello,', 'World!', 'My', 'name', 'is', 'John!']
```

```
str1.split(',') -> [' Hello', ' World! My name is John! ']  
str1.replace('John', 'Jeff') -> Hello, World! My name is Jeff!
```

Операции със стрингове.

Конкатенация

```
In [10]: greeting = 'Hello, '  
name = 'John'  
  
full_greeting = greeting + name + '!'  
  
print(full_greeting)
```

```
Hello, John!
```

Форматиране на стрингове

Форматирането на стрингове може да стане както с конкатенация, така и с `.format()`.

```
In [11]: text1 = 'Nice to meet you, John! My name is '  
text2 = ' and I\'m '  
text3 = ' years old.'  
  
name = 'Maria'  
age = 21  
  
full_text = text1 + name + text2 + str(age) + text3  
  
print(full_text)
```

```
Nice to meet you, John! My name is Maria and I'm 21 years old.
```

Конкатенацията работи, но `.format()` ни дава възможност за много по-чист код.

```
In [12]: full_text = 'Nice to meet you, John! My name is {} and I\'m {} years old!'  
  
name = 'Maria'  
age = 21  
  
print(full_text.format(name, age))
```

```
Nice to meet you, John! My name is Maria and I'm 21 years old!
```

F-strings

F-strings присъстват в Python от версия 3.6 и са модерният начин за форматиране на изход (а и форматиране на стрингове). След като се запознахме с `.format()`, работата с fstrings е също толкова

интуитивна.

```
In [13]: name = 'Maria'
age = 21

full_text = f'Nice to meet you, John! My name is {name} and I\'m {age} years old!'

print(full_text)
```

```
Nice to meet you, John! My name is Maria and I'm 21 years old!
```

Форматиране на вход/изход с .format() и fstrings

Тъй като вход/изхода в Python се осъществява чрез стрингове форматирането на вход/изход и това на стрингове са тясно свързани. Някои допълнителни подходи при форматиране на изхода са:

Брой знаци след запетаята

При fstring, както и при .format() по еднакъв начин може да настроим формата, в който извеждаме дадената променлива (в случая дробно число, но може да се приложи и за формат на дата, час и т.н.)

```
In [14]: pi = 3.1415926535

print(f'PI = {pi}')
print(f'PI = {pi:.5f}')
print(f'PI = {pi:.3f}')
print(f'PI = {pi:.1f}')
```

```
PI = 3.1415926535
PI = 3.14159
PI = 3.142
PI = 3.1
```

Ширина на полето

```
In [15]: number = 123

print(f'number = {number}')
```

```
print(f'number = {number: > 10}, and some other text')
print(f'number = {number: < 10}, and some other text')
```

```
number = 123
number =          123, and some other text
number = 123      , and some other text
```

Относно функцията input()...

Функцията `input()` в Python се използва за да въведем информация от някакъв `command-line interface` (CLI). Функцията по подразбиране връща стойност от тип `string`, т.е. е възможно да се наложи да извършим `typecasting`.

```
In [16]: username = input('Enter your username:')
age = int(input('Enter your age:'))
average_grade = float(input('Enter your average grade:'))

print(f'Hello, {username}! I see that you are {age} years old and \
your average grade is {average_grade}!')
```

```
Enter your username:John
Enter your age:21
Enter your average grade:5.77
Hello, John! I see that you are 21 years old and your average grade is 5.77!
```

Самостоятелни задачи

1. Напишете програма, която проверява дали дадена дума е палиндром.
2. Напишете програма, която проверява дали даден даден текст има четен брой символи.
3. Напишете програма, която проверява дали даден даден текст има четен брой съгласни.
4. Напишете програма, която изписва първите `n` на брой символа от даден текст.
5. Напишете програма, която принтира думите в текст, които имат нечетен брой символи.
6. Напишете програма, която конвертира `snake case` към `camel case`.